

Introduction to Cryptography

Refresh of basic math foundations for cryptography

Lei Xu

lxu12@kent.edu

Computer Science Department
Kent State University
September 13, 2026

Review roadmap

1. Notation and modular arithmetic: read and compute cipher expressions
2. Probability and uniform keys: describe random choices and guessing
3. XOR algebra and key reuse: simplify equations and explain leakage
4. Proofs and correctness: justify claims for every allowed input
5. Statistics and frequency analysis: connect probabilities to data

Notation and modular arithmetic

Messages, keys, and sets

$$m \in \mathcal{M}, \quad k \in \mathcal{K}, \quad c = \text{Enc}_k(m)$$

- $\mathcal{M}$ is the set of allowed messages, the message space
- $\mathcal{K}$ is the set of allowed keys, the key space
- $\in$ means “belongs to”
- $\text{Enc}_k(m)$ means encrypt message m with key k

Example: $\mathcal{M} = \{0, 1\}^8$ contains all 256 strings of eight bits.

Remainders and congruence

For an integer a and positive integer n , write

$$a = qn + r, \quad 0 \leq r < n. \quad (\text{This presentation is unique})$$

Then $a \bmod n = r$. For example,

$$35 \bmod 10 = 5, \quad 28 \bmod 26 = 2.$$

Congruence means that two integers differ by a multiple of n :

$$a \equiv b \pmod{n} \iff n \mid (a - b).$$

Thus $35 \equiv 5 \pmod{10}$, although $35 \neq 5$ as integers.

Addition and subtraction modulo 26

Reduce the result to one of $0, 1, \dots, 25$:

$$(24 + 5) \bmod 26 = 29 \bmod 26 = 3,$$

$$(2 - 5) \bmod 26 = (-3) \bmod 26 = 23.$$

- Subtract 26 from 29 to obtain 3
- Add 26 to -3 to obtain 23
- Both operations preserve the congruence class

The mathematical remainder here always lies between 0 and 25.

Indices and repeated keys

Use indices starting at 0. For a message of length n and a key of length ℓ ,

$$m = (m_0, \dots, m_{n-1}), \quad k = (k_0, \dots, k_{\ell-1}).$$

The expression $k_{i \bmod \ell}$ selects a key position cyclically.

Message position i	0	1	2	3	4	5	6
$i \bmod 3$	0	1	2	0	1	2	0
Key entry	k_0	k_1	k_2	k_0	k_1	k_2	k_0

Depending on the scheme, m_i may denote a bit, byte, or letter index.

Shift cipher and Vigenère cipher

Encode letters as $A = 0, B = 1, \dots, Z = 25$.

For a shift key k ,

$$c = (m + k) \bmod 26, \quad m = (c - k) \bmod 26.$$

Example: Y has value 24. With $k = 5$, it encrypts to D , value 3.

For a Vigenère key of length ℓ ,

$$c_i = (m_i + k_{i \bmod \ell}) \bmod 26.$$

The inner modulo selects a key position. The outer modulo wraps letter values.

Practice: modular arithmetic

1. Compute $(23 + 7) \bmod 26$ and $(1 - 4) \bmod 26$
2. For a key of length 4, which key entry encrypts m_{10} ? Use indices starting at 0
3. Encrypt YZAB with the repeating Vigenère key DF, whose values are $(3, 5)$

Write the numerical letter values before converting back to letters.

Practice solutions: modular arithmetic

1. $(23 + 7) \bmod 26 = 4$ and $(1 - 4) \bmod 26 = 23$
2. $10 \bmod 4 = 2$, so use k_2 , the third key entry
3. Repeat the key values and add modulo 26:

Plaintext	<i>Y</i>	<i>Z</i>	<i>A</i>	<i>B</i>
Message value	24	25	0	1
Key value	3	5	3	5
Sum modulo 26	1	4	3	6
Ciphertext	<i>B</i>	<i>E</i>	<i>D</i>	<i>G</i>

Probability and uniform keys

Events and the complement rule

An **event** is a set of outcomes of a random experiment. Its probability, $\Pr[E]$, is a number between 0 and 1.

For a fair six-sided die, let E be the event “roll a 4”:

$$\Pr[E] = \frac{1}{6}.$$

The **complement** $\neg E$ means that E does not occur:

$$\Pr[\neg E] = 1 - \Pr[E], \quad \Pr[\text{not rolling a 4}] = 1 - \frac{1}{6} = \frac{5}{6}.$$

- Probability 0 means impossible in this finite experiment
- Probability 1 means certain

Independent events

Two events are **independent** when learning that one occurred does not change the probability of the other.

$$\Pr[A \wedge B] = \Pr[A] \Pr[B]$$

Here $A \wedge B$ means that *both* events occur.

For two independent fair coin flips,

$$\Pr[\text{two heads}] = \frac{1}{2} \cdot \frac{1}{2} = \frac{1}{4}.$$

- Multiplying the individual probabilities requires independence
- If the second result copies the first, two heads has probability $1/2$, even though each individual result is equally likely to be heads or tails

Random variables and their values

A **random variable** assigns a value to the outcome of a random experiment.

- Examples include a random bit B , a random key K , or a randomly chosen nonce N
- Uppercase K denotes the random variable, while lowercase k denotes a particular key value
- $\Pr[K = k]$ is the probability that the chosen key equals that value

We write a uniform choice of a 128-bit key as

$$K \xleftarrow{\$} \{0, 1\}^{128}.$$

The set contains all 128-bit strings. The dollar sign marks a uniform random choice.

Choosing a key uniformly at random

Sampling a key means choosing a key at random. **Uniformly** means every allowed key has the same chance of being chosen.

$$\Pr[\text{a particular outcome}] = \frac{1}{\text{number of possible outcomes}}.$$

For a two-bit key, the four possible choices are:

Key	00	01	10	11
Probability	1/4	1/4	1/4	1/4

- Imagine four identical slips, one key on each, mixed in a bag
- Draw one slip without looking and use the key written on it
- Uniform describes the chances before the draw, not the appearance of the key afterward

Uniform bit-string keys

To choose an n -bit key uniformly, imagine n independent fair coin flips. Record heads as 0 and tails as 1.

$$\Pr[\text{any particular } n\text{-bit key}] = \underbrace{\frac{1}{2} \times \cdots \times \frac{1}{2}}_{n \text{ factors}} = \frac{1}{2^n}.$$

- There are 2^n possible keys, and each has the same probability
- Repeated bits are allowed: 00 and 01 each have probability $1/4$ for a two-bit key
- Requiring equal numbers of zeros and ones would exclude some keys and change the distribution

Quick check: Flip one fair coin and repeat its result twice. Does this choose uniformly from all four two-bit keys?

Guessing a uniformly chosen key

Suppose $K \xleftarrow{\$} \{0, 1\}^{128}$. For every particular key k ,

$$\Pr[K = k] = \frac{1}{2^{128}}.$$

- One guess made without information about K succeeds with probability 2^{-128} , whether the guess is fixed or chosen independently at random
- Trying q distinct candidate keys covers a fraction $q/2^{128}$ of the key space, for $0 \leq q \leq 2^{128}$
- Even 2^{40} distinct guesses cover only a fraction 2^{-88} of all possible keys

These are probabilities for guessing the key. Other attacks depend on the scheme.

XOR algebra and key reuse

XOR on bits and bit strings

XOR, written $\oplus$, outputs 1 exactly when its two input bits differ.

x	y	$x \oplus y$
0	0	0
0	1	1
1	0	1
1	1	0

Apply XOR separately at each position of equal-length bit strings:

$$1011 \oplus 0110 = 1101.$$

On individual bits, XOR is addition modulo 2, with no carry.

Algebraic properties of XOR

For equal-length bit strings x, y, z , let 0 denote the all-zero string.

$$x \oplus 0 = x$$

identity

$$x \oplus x = 0$$

self-cancellation

$$x \oplus y = y \oplus x$$

commutativity

$$(x \oplus y) \oplus z = x \oplus (y \oplus z)$$

associativity

These rules let us reorder and regroup terms, then cancel pairs.

Encryption and decryption with XOR

For equal-length message m and key k ,

$$c = m \oplus k, \quad m = c \oplus k.$$

	Encryption	Decryption
Input	1011	1101
Key	0110	0110
Output	1101	1011

XORing with the same key twice recovers the message.

Known plaintext reveals the matching key bits

Suppose the attacker knows c and its corresponding plaintext m .

$$\begin{aligned}c \oplus m &= (m \oplus k) \oplus m \\&= (m \oplus m) \oplus k \\&= k.\end{aligned}$$

- Known plaintext reveals the key bits used at those positions
- With a repeated key, those bits also decrypt other positions that use the same key entries
- Independent keys for other messages do not follow from this equation

Reusing a key cancels it out

Assume two equal-length messages use the same key at the same positions:

$$c_1 = m_1 \oplus k, \quad c_2 = m_2 \oplus k.$$

An attacker can compute

$$\begin{aligned} c_1 \oplus c_2 &= (m_1 \oplus k) \oplus (m_2 \oplus k) \\ &= (m_1 \oplus m_2) \oplus (k \oplus k) \\ &= m_1 \oplus m_2. \end{aligned}$$

The ciphertexts reveal a relation between the messages without revealing k .

What the XOR of two messages reveals

Let $d = c_1 \oplus c_2 = m_1 \oplus m_2$ for n -bit messages.

- A zero bit in d means the corresponding plaintext bits are equal
- A one bit means they differ
- If m_1 is known, then $m_2 = d \oplus m_1$
- Without more information, each candidate m_1 determines a candidate $m_2 = d \oplus m_1$

There are 2^n candidate pairs before applying restrictions such as language or file format. The relation alone does not uniquely determine either message.

Practice: XOR and key reuse

1. Simplify $x \oplus y \oplus x \oplus z \oplus y$
2. Encrypt $m = 0101\ 0111$ using $k = 1111\ 1111$
3. Two ciphertexts use the same aligned key:

$$c_1 = 1100, \quad c_2 = 1010.$$

If $m_1 = 0110$, what is m_2 ?

For each answer, identify the XOR property you used.

Practice solutions: XOR and key reuse

1. Reorder and cancel the repeated terms:

$$(x \oplus x) \oplus (y \oplus y) \oplus z = z$$

2. XOR each bit:

$$0101\ 0111 \oplus 1111\ 1111 = 1010\ 1000$$

3. First cancel the reused key, then cancel m_1 :

$$m_2 = c_1 \oplus c_2 \oplus m_1 = 1100 \oplus 1010 \oplus 0110 = 0000$$

Proofs and correctness

Quantifiers and logical statements

$\forall$ “For every.” A claim must hold for each allowed value

$\exists$ “There exists.” At least one allowed value works

$P \Rightarrow Q$ If P holds, then Q holds

$P \Leftrightarrow Q$ Both $P \Rightarrow Q$ and $Q \Rightarrow P$ hold

For a deterministic encryption scheme, correctness means

$$\forall k \in \mathcal{K}, \forall m \in \mathcal{M}, \quad \text{Dec}_k(\text{Enc}_k(m)) = m.$$

One successful example does not establish a claim about every input.

Quantifier order changes the claim

For $n \geq 1$, compare these statements about n -bit strings:

$$\forall m \exists k : \quad m \oplus k = 0.$$

This is true: for each message, choose $k = m$.

$$\exists k \forall m : \quad m \oplus k = 0.$$

This is false: one fixed key cannot equal every message.

The order tells us whether the key may depend on the message.

Structure of a direct proof

1. State the claim and its assumptions
2. Choose arbitrary inputs from the allowed sets
3. Substitute definitions and justify each step
4. State why the result establishes the claim for all allowed inputs

For encryption correctness, start with $\text{Dec}_k(\text{Enc}_k(m))$ and show that it equals m .

A proof of XOR decryption correctness

Claim. For every $m, k \in \{0, 1\}^n$, XOR decryption recovers m .

Proof. Fix arbitrary m and k . Define $\text{Enc}_k(m) = m \oplus k$ and $\text{Dec}_k(c) = c \oplus k$. Then

$$\begin{aligned}\text{Dec}_k(\text{Enc}_k(m)) &= (m \oplus k) \oplus k && \text{definitions} \\ &= m \oplus (k \oplus k) && \text{associativity} \\ &= m \oplus 0 && \text{self-cancellation} \\ &= m && \text{identity.}\end{aligned}$$

The argument holds for every allowed m and k .

Correctness of modular decryption

Let $m, k \in \{0, \dots, 25\}$ and $c = (m + k) \bmod 26$. By definition,

$$c \equiv m + k \pmod{26}.$$

Subtract k from both sides:

$$c - k \equiv m \pmod{26}.$$

Since m is already in $\{0, \dots, 25\}$,

$$(c - k) \bmod 26 = m.$$

For Vigenère, apply this argument at every position with key entry $k_{i \bmod \ell}$.

Correctness and secrecy are different claims

The correctness proof works for *every* key. Consider a scheme that always chooses the all-zero key:

$$c = m \oplus 0 = m.$$

- Correctness asks whether the receiver recovers the message
- Secrecy asks what the attacker learns from the ciphertext
- A one-time pad chooses its secret key with equal probability among all bit strings of the message's length, independently of the message, and uses it only once

An algebraic correctness proof alone does not establish secrecy.

Practice: a proof and a counterexample

1. Prove that, for equal-length bit strings,

$$x \oplus k = y \oplus k \quad \Rightarrow \quad x = y.$$

Write the operation you apply to both sides

2. One claims: “If decryption always works, the scheme hides the message.” Give a counterexample

A proof establishes a universal claim. One counterexample can refute it.

Practice solutions: proof and counterexample

Cancellation proof. XOR both sides with k :

$$(x \oplus k) \oplus k = (y \oplus k) \oplus k,$$

$$x \oplus (k \oplus k) = y \oplus (k \oplus k),$$

$$x = y.$$

Counterexample. Define

$$\text{Enc}_k(m) = m, \quad \text{Dec}_k(c) = c.$$

Decryption always recovers the message, but the ciphertext reveals it directly.

Statistics and frequency analysis

Probability and observed frequency

For a symbol a , distinguish

$$p_a = \Pr[X = a] \quad \text{and} \quad q_a = \frac{\text{number of occurrences of } a}{\text{sample length}}.$$

- p_a describes a probability model
- q_a describes the sample we observed
- In a uniform model on r symbols, every $p_a = 1/r$
- A fair coin gives heads and tails equal chances, but 10 flips might produce 7 heads and 3 tails: $p_{\text{heads}} = 1/2$, while $q_{\text{heads}} = 7/10$

Natural language has unequal symbol frequencies and dependencies between symbols.

Reading a summation

If q_i is the frequency of symbol i , then

$$\sum_{i=0}^{r-1} q_i^2 = q_0^2 + q_1^2 + \cdots + q_{r-1}^2.$$

- The index i runs over the r symbols
- Square each frequency, then add the results

For three symbols with frequencies $\frac{1}{2}, \frac{1}{4}, \frac{1}{4}$,

$$\sum_{i=0}^2 q_i^2 = \left(\frac{1}{2}\right)^2 + \left(\frac{1}{4}\right)^2 + \left(\frac{1}{4}\right)^2 = \frac{3}{8}.$$

Later, we will interpret this as a probability of matching symbols.

Matching symbols and squared probabilities

Let X and Y be independent draws from the same distribution (p_i) .

$$\Pr[X = Y] = \sum_i \Pr[X = i, Y = i] = \sum_i p_i^2.$$

- Independence gives $\Pr[X = i, Y = i] = p_i^2$
- For r equally likely symbols, the result is $r(1/r)^2 = 1/r$
- For $(\frac{1}{2}, \frac{1}{4}, \frac{1}{4})$, the result is $\frac{3}{8} > \frac{1}{3}$

Using observed frequencies q_i , $\sum_i q_i^2$ is the matching probability when sampling twice independently *with replacement* from the sample.

Why repeated keys permit frequency analysis

Suppose the key length is ℓ . Group positions by $i \bmod \ell$.

- Every position in one group uses the same key entry
- A fixed Vigenère shift permutes letter labels but preserves counts
- XOR with a fixed byte similarly permutes the 256 byte values
- With enough data, an attacker can compare candidate decryptions with expected language patterns

Short samples and incorrect key-length guesses can make these comparisons unreliable.

Review questions

1. What does $k_i \bmod \ell$ mean, and why does it repeat?
2. Which XOR identities justify decryption and key cancellation?
3. Why does one successful test fail to prove correctness for all inputs?
4. Why does modular decryption recover a letter value in $0, \dots, 25$?
5. Why can repeated-key encryption preserve exploitable statistics?

In a written solution, state the assumptions, show the intermediate steps, and explain exactly what the conclusion establishes.